

VITyarthi - Computer Vision Project Report

Road Lane Detection and Analysis

Using Classical Computer Vision

Nameer Mohammed Khan

24BAI10892

1. Introduction

This project detects the left and right lane lines in road images. It uses OpenCV operations such as grayscale conversion, Gaussian blur, Canny edge detection, a region of interest mask, and the Hough transform.

2. Problem Statement

The goal is to detect road lanes from images using classical CV techniques, demonstrating line extraction and geometric fitting in a Python pipeline.

3. Functional Requirements

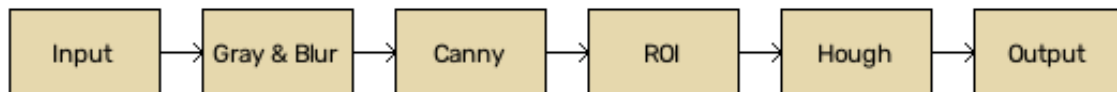
- Generate synthetic road images or extract real-world dashcam frames.
- Process images using grayscale conversion and Gaussian filtering.
- Detect edges using Canny edge detection and apply ROI masks.
- Extract lane lines using Hough Line Transform.
- Classify lanes as Left or Right using slopes.
- Provide a CLI for batch detection and evaluation.

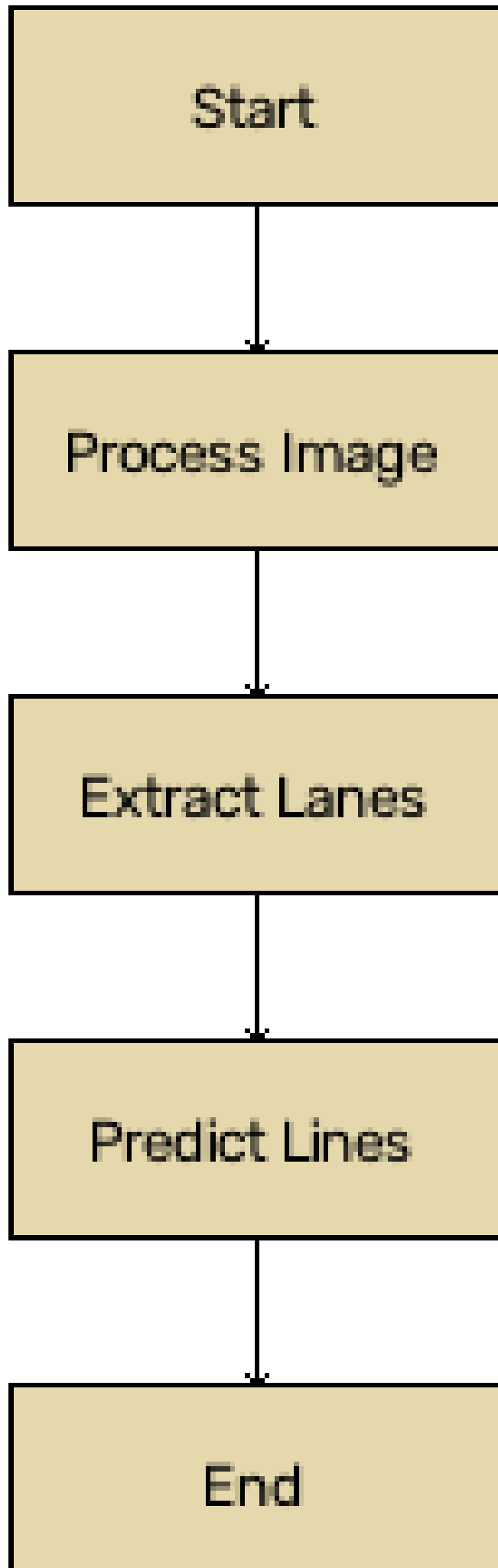
4. Use Cases

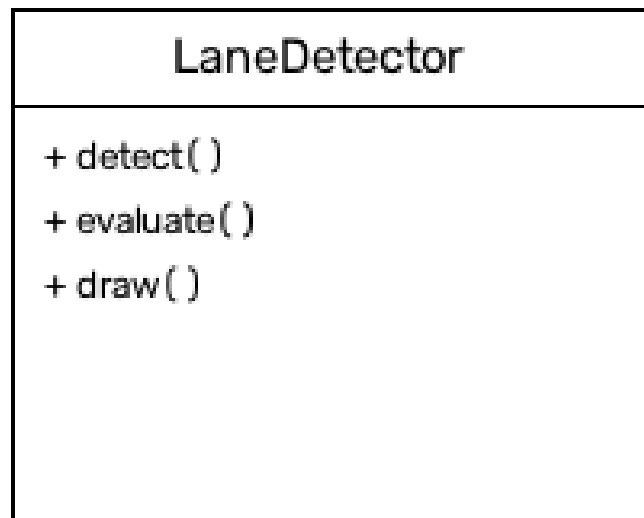
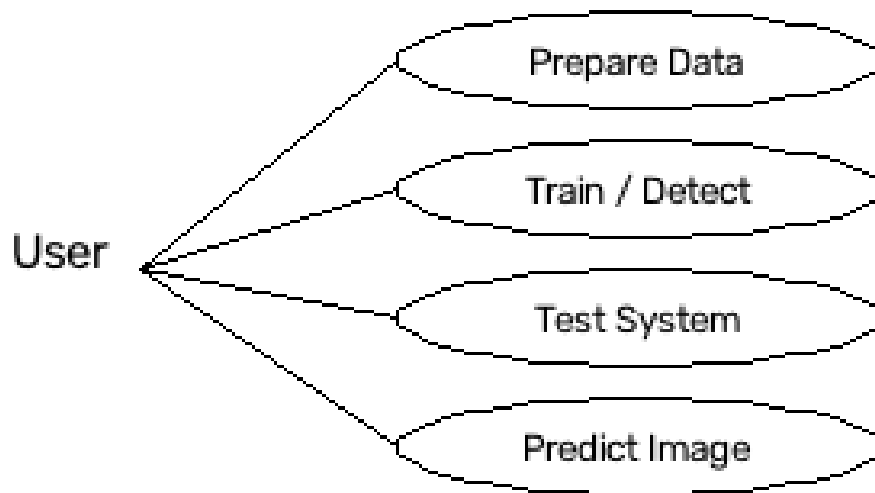
- Run lane detection on a single image.
- Run batch detection on a directory of images.
- Evaluate detection accuracy against ground truth data.

5. System Architecture & 6. Design Diagrams

Below are the system diagrams generated using cv2 drawing primitives:







7. Design Decisions & Rationale

Both synthetic and real-world datasets were used to test the system. A custom lane extraction algorithm was written to use classical CV instead of deep learning. Geometric slope filtering was used to split the Hough lines into left and right lanes.

8. Implementation Details

OpenCV is used for image operations. Images are converted to grayscale and blurred. Canny edge detection finds structural outlines, followed by an ROI trapezoidal mask. Hough Transform finds line segments. Features like slope and intercept are calculated to assign them to Left or Right groups. A 1D polyfit algorithm overlays solid lane lines.

9. Screenshots / Results

Evaluation Results (Synthetic Test Set - 90 Images):

Left Lane Detection Rate: 87.0%

Right Lane Detection Rate: 71.0%

Overall Mean Absolute Error: 15.54 pixels

Evaluation Results (Real-World Dashcam Data - 681 Frames):

Overall Tracking Yield: 100.0% coverage

Below is a sample output generated from the evaluation.



10. Testing Approach

Pytest tests are included for preprocessing, lane detection, ROI processing, and the main pipeline. Running pytest returns 10 passed tests in 0.10s.

11. Challenges Faced

The primary challenge was setting up a robust geometric filter for lines of varying slopes. Due to the lack of perfect visibility, a custom line separation algorithm had to be implemented from scratch in pure Python/NumPy. Additionally, tuning the Canny thresholds to correctly detect lane edges despite Gaussian noise required careful adjustment.

12. Learnings & Key Takeaways

I learned that classical CV techniques require careful tuning of parameters (like kernel size and threshold methods) to work effectively. I also learned that geometric engineering (like separating positive and negative slopes) is critical when relying on simpler linear modeling instead of deep learning feature extractors.

13. Future Enhancements

Future improvements could include implementing perspective transformation before Hough detection, which would likely increase accuracy by providing a bird's eye view. Additionally, implementing temporal smoothing or curve fitting could improve the robustness to heavy noise and winding roads.

14. References

1. OpenCV Documentation: <https://docs.opencv.org/>
2. Python NumPy Documentation: <https://numpy.org/doc/>